

Mini-projet – Classification de textes

Comparer trois approches sur un cas d'usage personnel

Travail	Individuel ou en groupe
Barème	20 points
À rendre	un notebook exécutable et un court rapport PDF

Objectif

Choisissez un **cas d'usage de classification de textes** qui vous est propre : thème, langue, domaine et classes sont libres. Exemples possibles : catégorisation de demandes, détection d'intention, modération, sentiment, identification d'un registre ou classement de documents.

Sur les **mêmes données** et avec le **même protocole d'évaluation**, vous comparerez trois systèmes :

1. une méthode **sans apprentissage automatique** ;
2. un modèle de **deep learning** entraîné par vos soins ;
3. un **Transformer pré-entraîné pour la classification**.

Le but n'est pas d'obtenir le meilleur score possible, mais de construire une comparaison claire, honnête et reproductible.

Travail demandé

1. Cas d'usage et données

Présentez le besoin, les classes et la provenance des données. Le jeu de données doit contenir au moins **deux classes** et suffisamment d'exemples pour permettre une séparation en ensembles d'entraînement, de validation et de test. Vérifiez la distribution des classes, les doublons et quelques exemples bruts.

2. Système sans machine learning

Construisez une baseline déterministe à partir de règles explicites : mots-clés, expressions régulières, lexique, scores ou heuristiques propres au cas d'usage. Expliquez les règles et leurs limites.

3. Modèle de deep learning

Implémentez et entraînez un réseau simple avec PyTorch. Une architecture minimale est attendue, par exemple : représentation des textes, couche cachée avec activation non linéaire, puis couche de classification. Un MLP sur des vecteurs TF-IDF ou un petit réseau à embeddings est acceptable. Affichez les pertes d'entraînement et de validation et justifiez les principaux hyperparamètres.

4. Transformer de classification

Utilisez un Transformer pré-entraîné adapté à la langue et une tête de classification. Effectuez au minimum l'entraînement de la tête de classification ; vous pouvez affiner davantage le modèle si les ressources le permettent. Documentez le modèle choisi, la tokenisation, la longueur maximale et les paramètres d'entraînement.

5. Évaluation commune

Choisissez les modèles et hyperparamètres sur la **validation**. Le test ne doit être utilisé qu'à la fin. Pour les trois systèmes, présentez au minimum :

- accuracy, précision, rappel et F1 macro ;
- matrice de confusion ;
- temps d'entraînement et d'inférence, lorsque pertinent ;
- analyse commentée d'au moins cinq erreurs.

Terminez par une comparaison : quel système retiendriez-vous pour votre cas d'usage, et pourquoi ? Prenez en compte les scores, le coût, la simplicité et les limites observées.

Contraintes de méthode

- Une même définition des classes et un même jeu de test pour les trois systèmes.
- Aucun apprentissage du vocabulaire, des règles ou des hyperparamètres sur le test.
- Seed aléatoire fixée.
- Code lisible, cellules exécutables dans l'ordre et résultats visibles.

Livrables

- `projet.ipynb`, exécuté et autonome autant que possible ;
- `rapport.pdf`, de **4 pages maximum** hors annexes ;
- éventuellement un petit fichier de données ou un script de téléchargement ;
- `README.txt`, avec les instructions d'exécution et les dépendances.

Le rapport doit décrire le cas d'usage, les données, les trois méthodes, les résultats et la conclusion. Les sorties volumineuses et le code complet restent dans le notebook.

Barème (à titre intégralement indicatif)

Critère	Points
Cas d'usage, données et protocole expérimental	3
Baseline sans machine learning : pertinence et explicitation	3
Modèle de deep learning : implémentation et entraînement	4
Transformer de classification : choix et implémentation	4
Évaluation comparative et analyse d'erreurs	4
Reproductibilité, clarté du code et qualité du rapport	2
Total	20

Un bon projet ne se limite pas au score : il rend visibles les hypothèses, les compromis et les erreurs de chaque approche.